# Programación en Java para Algoritmos y Estructuras de Datos

## Parte I

Rodrigo Pérez Rodríguez

Universidad Rey Juan Carlos

# Contenido


1. Introducción
2. Arquitectura y modelo de programación
3. Ejemplo: Filtrado con lista blanca
4. Algoritmos de búsqueda y eficiencia
5. Componentes de un programa
6. Abstracción de datos y modularidad
7. Discusión

# Introducción

# Sentencias elementales

- Declaraciones de datos: dan nombre a zonas de memoria (ej. `int i;`, `double[] velocidad = {2.3, 4.45, 100.01};`)
- Sentencias de asignación: guardan resultados de expresiones (ej. `y = 3 * (x + 1);`, `y += 1;`)
- Sentencias de control de flujo: alternativas (`if`, `switch`) y repetitivas (`while`, `for`)

**Importante**

Cualquier programa secuencial puede escribirse solo con estas sentencias.

# Mecanismos adicionales

- Procedimientos y funciones: abstracción funcional y reutilización
  - **Función:** devuelve un valor (ej. `double sqrt(double c)`)
  - **Procedimiento:** no devuelve valor, tipo `void` (ej. `void imprimir(int x)`)
  - Ambos pueden tener parámetros formales
- Clases y objetos: tipos abstractos de datos (encapsulan variables y métodos)
- Módulos: organización y agrupación de código

**Nota**

Estos mecanismos permiten manejar la complejidad de programas grandes, pero no tienen reflejo directo en el código máquina.

# Arquitectura y modelo de programación

# ¿Por qué no pseudocódigo?

**Los algoritmos son independientes del lenguaje de programación**

- Un algoritmo es una secuencia de pasos para resolver un problema
- Puede expresarse en cualquier lenguaje: Java, Python, C++, JavaScript, etc.
- La idea del algoritmo es independiente de los detalles de implementación

**Limitaciones del pseudocódigo:**

- **Incompleto:** solo expresa ideas abstractas, no programas ejecutables
- **No ejecutable:** no se pueden probar ni experimentar con él
- **Ambiguo:** cada autor usa su propia notación
- **No aplicable:** no se puede reutilizar en aplicaciones reales

**Conclusión**

Necesitamos un lenguaje de programación real para expresar, probar y aplicar algoritmos

# Modelo de programación

- Subconjunto de Java utilizado en la asignatura: tipos primitivos, arrays, métodos estáticos, entrada/salida
- Bibliotecas estándar y específicas (algs4.jar)
- Funcionalidad del sistema operativo
- Permite expresar algoritmos sin oscurecer su comprensión, pero con robustez

# Ventajas de usar Java

- Expresión concisa y completa de algoritmos
- Ejecución y experimentación: permite estudiar propiedades experimentalmente
- Reutilización en aplicaciones reales
- Facilita la detección de errores mediante tipos
- Fácil traslado a otros lenguajes

# Ejemplo: Filtrado con lista blanca

# Ejemplo: LinearSearch.java

```java
import java.util.Arrays;
import edu.princeton.cs.algs4.In;
import edu.princeton.cs.algs4.StdIn;

public class LinearSearch {
    public static int indexOf(int key, int[] a) {
        for (int i = 0; i < a.length; i++)
            if (a[i] == key) return i;
        return -1;
    }
    public static void main(String[] args) {
        In in = new In(args[0]);
        int[] whitelist = in.readAllInts();
        while (!StdIn.isEmpty()) {
            int key = StdIn.readInt();
            if (indexOf(key, whitelist) == -1)
                System.out.println(key);
        }
    }
}
```

# Explicación del ejemplo

- El método `indexOf` busca un número en un array.
- Devuelve el índice si lo encuentra, o -1 si no está.
- Se usa para filtrar números según una lista blanca.
- Implementa búsqueda lineal: compara secuencialmente cada elemento.
- Útil para listas pequeñas, pero ineficiente para grandes volúmenes de datos.

# Compilación y ejecución

```
$ javac -cp .:algs4.jar LinearSearch.java
$ java  -cp .:algs4.jar LinearSearch algs4-data/largeW.txt < algs4-data/largeT.txt
499569
984875
...
32449528
38862597
69830567
```

# ¿Qué sucede al compilar en Java?

**Proceso de compilación:**

- `javac` compila `LinearSearch.java` → genera `LinearSearch.class`
- El fichero `.class` contiene **bytecode**, no código máquina nativo

**¿Qué es el bytecode?**

- Instrucciones para la **Máquina Virtual de Java (JVM)**, no para el microprocesador
- Es **portable**: el mismo .class funciona en Windows, Linux, Mac, etc.
- La JVM traduce bytecode a código máquina del microprocesador real

**Ejecución (Just in Time - JIT):**

- `java` arranca la JVM que lee el .class
- La JVM traduce bytecode a código nativo **durante la ejecución**
- No traduce todo al inicio: va traduciendo "justo a tiempo" cuando se necesita ejecutar cada parte

# Discusión

**Pregunta:** ¿Por qué es tan lento este programa con listas grandes?

- La búsqueda lineal compara cada elemento uno por uno.
- Para $10^7$ búsquedas en una lista de $10^6$ elementos, puede tomar más de 45 minutos.
- Necesitamos algoritmos más eficientes para grandes volúmenes de datos.

# Algoritmos de búsqueda y eficiencia

# Mejora: búsqueda lineal en array ordenado (1)

```
import edu.princeton.cs.algs4.In;
import edu.princeton.cs.algs4.StdIn;
import java.util.Arrays;

public class SortedLinearSearch {
    public static int indexOf(int key, int[] a) {
        for (int i = 0; i < a.length; i++) {
            if (a[i] == key)
                return i;
            else if (a[i] > key)
                return -1;
        }
        return -1;
    }
```

# Mejora: búsqueda lineal en array ordenado (2)

```
    public static void main(String[] args) {
        In in = new In(args[0]);
        int[] whitelist = in.readAllInts();

        Arrays.sort(whitelist);  // Ordena el array (método estático)
        while (!StdIn.isEmpty()) {
            int key = StdIn.readInt();
            if (indexOf(key, whitelist) == -1)
                System.out.println(key);
        }
    }
}
```

# Clases estáticas

- `Arrays.sort(whitelist);` llama a un método estático de la clase `Arrays`
- No necesitamos crear un objeto: `Arrays` es una clase con métodos estáticos
- **Ventaja:** Termina antes si encuentra un elemento mayor que el buscado
- **Limitación:** Aún así, sigue siendo lineal en el peor caso ($O(n)$)

# Búsqueda binaria: BinarySearch.java (1)

```
import edu.princeton.cs.algs4.In;
import edu.princeton.cs.algs4.StdIn;
import java.util.Arrays;

public class BinarySearch {
    public static int indexOf(int key, int[] a) {
        int lo = 0, hi = a.length - 1;
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2;
            if (key < a[mid]) hi = mid - 1;
            else if (key > a[mid]) lo = mid + 1;
            else return mid;
        }
        return -1;
    }
```

# Búsqueda binaria: BinarySearch.java (2)

```
    public static void main(String[] args) {
        In in = new In(args[0]);
        int[] whitelist = in.readAllInts();
        Arrays.sort(whitelist);
        while (!StdIn.isEmpty()) {
            int key = StdIn.readInt();
            if (indexOf(key, whitelist) == -1)
                System.out.println(key);
        }
    }
}
```

# Comparativa de tiempos

- Búsqueda lineal: más de 45 minutos para $10^7$ búsquedas en lista de $10^6$
- Búsqueda binaria: menos de 15 segundos para el mismo caso
- La búsqueda binaria requiere que el array esté ordenado previamente

**Pregunta:** ¿Por qué la diferencia es tan grande?

- Complejidad: lineal ($O(n)$) vs. logarítmica ($O(\log n)$)
- Para $n = 10^6$, $\log_2 n \approx 20$ comparaciones vs. hasta $10^6$

# Componentes de un programa

# Estructura de un programa Java

**En Java un programa es una clase**

- Cualquier código que se escribe en Java ha de estar dentro de una clase
- Una clase puede ser:
  - Una **biblioteca de métodos estáticos** (funciones)
  - La definición de un **tipo abstracto de datos**

# Componentes de un programa

**En una clase se pueden utilizar los siguientes 7 componentes:**

1. Tipos primitivos de datos y expresiones
2. Sentencias
3. Arrays
4. Métodos estáticos
5. Strings
6. Entrada/Salida
7. Abstracción de datos

# Tipos primitivos en Java

| | |
|---|---|
| int | Números enteros (ej. 42, -17) |
| double | Números reales (ej. 3.14, -2.5) |
| boolean | Valores lógicos true/false |
| char | Caracteres individuales (ej. 'a', 'Z', '5') |

# Ejemplos y operadores

| | |
|---|---|
| `+ - * / %` | Operadores aritméticos (suma, resta, multiplicación, división, módulo) |
| `< > <= >= == !=` | Comparaciones (menor, mayor, igual, diferente) |
| `&& \|\| !` | Operadores lógicos (AND, OR, NOT) |

# Conversión de tipos y casting

- Promoción automática: de int a double, etc., para evitar pérdida de precisión
- Casting explícito: `(int) 3.7` es 3 (trunca la parte decimal)
- Comparaciones producen booleanos: `x < 5` evalúa a true o false
- Operadores lógicos combinan booleanos: `(x > 0) && (y < 10)`

# Sentencias en Java

- Declaraciones: `int x;` reserva memoria para una variable
- Asignaciones: `x = 5;` guarda un valor
- Condicionales: `if (x < 0) x = -x;` ejecuta código condicionalmente
- Bucles: `while`, `for` repiten sentencias
- Llamadas a métodos: `System.out.println(x);`
- `return`: termina un método y devuelve un valor

# Ejemplo: condicional y bucle

```
if (x < 0) x = -x;
for (int i = 1; i <= N; i++)
    sum += 1.0/i;
```

# Creación e inicialización de arrays

```
final int N = 10;
double[] a = new double[N];  // Array de N elementos, inicializados a 0.0
for (int i = 0; i < N; i++)
    a[i] = 0.0;
int[] b = {1, 1, 2, 3, 5, 8};  // Inicialización literal
```

- Arrays tienen longitud fija una vez creados
- Acceso mediante índices: `a[0]` es el primer elemento
- Índices van de 0 a `length-1`

# Aliasing y arrays multidimensionales

- Asignar un array a otro copia la referencia (aliasing): `int[] c = b;` hace que c y b referencien el mismo array
- Modificar `c[0]` también modifica `b[0]`
- Arrays de dos dimensiones: `double[][] matrix = new double[3][4];`
- Acceso: `matrix[i][j]` para fila i, columna j
- Cada fila es un array independiente

# Definición de un método estático

```
public static double sqrt(double c) {
    if (c < 0) return Double.NaN;
    double err = 1e-15;
    double t = c;
    while (Math.abs(t - c/t) > err * t)
        t = (c/t + t) / 2.0;
    return t;
}
```

- Método que calcula la raíz cuadrada usando el método de Newton
- Parámetros: `double c` (valor de entrada)
- Valor de retorno: `double` (la raíz cuadrada)
- Llamada: `double raiz = sqrt(9.0);` devuelve 3.0

# Propiedades de los métodos

- Paso de parámetros por valor: se copia el valor, no la referencia
- Arrays: se pasa la referencia (se puede modificar el contenido dentro del método)
- Sobrecarga: métodos con mismo nombre pero diferentes parámetros
- `return` termina el método inmediatamente
- Métodos `void` pueden tener efectos laterales (modificar variables globales o imprimir)

# Recursión

- Un método puede llamarse a sí mismo
- Es importante definir un caso base para evitar recursión infinita
- Ejemplo: Fibonacci recursivo: $F(n) = F(n-1) + F(n-2)$, con $F(0) = 0$, $F(1) = 1$
- Ventajas: expresividad, claridad para problemas recursivos
- Desventajas: puede ser ineficiente (muchas llamadas) y causar stack overflow

# Ejemplo: Fibonacci.java

```java
public class Fibonacci {
    public static long fibonacci(int n) {
        if (n == 0) return 0;
        if (n == 1) return 1;
        return fibonacci(n-1) + fibonacci(n-2);
    }
    public static void main(String[] args) {
        for (int n = 0; n < 90; n++)
            System.out.println(n + " " + fibonacci(n));
    }
}
```

# Abstracción de datos y modularidad

# Ejemplo: StaticSETofInts.java

```java
public class StaticSETofInts {
    private int[] a;
    public StaticSETofInts(int[] keys) {
        a = new int[keys.length];
        for (int i = 0; i < keys.length; i++)
            a[i] = keys[i];
        Arrays.sort(a);  // Ordena el array para búsqueda binaria
    }
    public boolean contains(int key) { return indexOf(key) != -1; }
    private int indexOf(int key) {
        // Implementación de búsqueda binaria
        int lo = 0, hi = a.length - 1;
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2;
            if (key < a[mid]) hi = mid - 1;
            else if (key > a[mid]) lo = mid + 1;
            else return mid;
        }
        return -1;
    }
}
```

- Clase que representa un conjunto de enteros inmutable
- Constructor ordena los elementos para búsqueda eficiente
- Método `contains` usa búsqueda binaria privada

# Ejemplo: Whitelist.java

```
public class Whitelist {
    public static void main(String[] args) {
        In in = new In(args[0]);  // Lee archivo de lista blanca
        int[] a = in.readAllInts();  // Lee todos los enteros
        StaticSETofInts set = new StaticSETofInts(a);  // Crea conjunto
        while (!StdIn.isEmpty()) {
            int key = StdIn.readInt();  // Lee de entrada estándar
            if (!set.contains(key))    // Si no está en conjunto
                System.out.println(key);   // Imprime
        }
    }
}
```

# Ejemplo: Whitelist.java (continuación)

- Usa la clase `StaticSETofInts` para filtrado eficiente
- Lee lista blanca desde archivo pasado como argumento
- Lee números de entrada estándar y filtra los no permitidos
- Demuestra abstracción de datos: oculta implementación de búsqueda

# Ejemplo: PruebaError.java

```java
public class PruebaError {
    public static void main(String[] args) {
        int x = -3;
        System.out.println("Programa PruebaError");
        System.err.println("Traza: contenido de x: " + x);
        if (x < 0) {
            System.err.println("Error: el valor de x no puede ser negativo");
            System.exit(1);  // Termina con código de error
        }
    }
}
```

- `System.out`: salida estándar (pantalla)
- `System.err`: salida de error (también pantalla, pero separable)
- `System.exit(1)`: termina el programa con código de error
- Útil para depuración y manejo de errores

# Discusión

# Preguntas para discusión

- ¿Por qué es importante elegir el algoritmo adecuado?
- ¿Qué ventajas tiene la modularidad y la abstracción de datos?
- ¿Qué ocurre si accedemos fuera de los límites de un array?
- ¿Qué diferencias hay entre pasar variables primitivas y arrays a un método?
- ¿Cómo afectan los tipos de datos a la robustez de los programas?

# ¿Por qué es importante elegir el algoritmo adecuado?

- **Corrección:** El algoritmo debe resolver el problema correctamente
- **Eficiencia:** Tiempo de ejecución y uso de memoria son críticos
- **Ejemplo:** Para ordenar, quicksort es más eficiente que bubble sort en la mayoría de casos
- **Trade-off:** A veces algoritmos más lentos son más simples y correctos

# ¿Qué ventajas tiene la modularidad y la abstracción de datos?

- **Reutilización:** Código modular se puede usar en múltiples programas
- **Mantenibilidad:** Cambios se localizan en módulos específicos
- **Legibilidad:** Código dividido en partes lógicas y comprensibles
- **Depuración:** Errores se aíslan más fácilmente

# ¿Qué ocurre si accedemos fuera de los límites de un array?

- Se lanza `ArrayIndexOutOfBoundsException`
- **Causa:** Intentar acceder a índice negativo o >= longitud del array
- **Ejemplo:** `int[] a = new int[5]; a[5] = 10;` // Error
- **Prevención:** Siempre verificar límites antes de acceder

# ¿Qué diferencias hay entre pasar variables primitivas y arrays?

- **Primitivas:** Se pasan por valor (copia del valor)
- **Arrays:** Se pasan por referencia (puntero al array original)
- **Consecuencia:** Modificar un array parámetro afecta al original
- **Ejemplo:** `void modificar(int x, int[] arr)` - x no cambia, arr sí

# ¿Cómo afectan los tipos de datos a la robustez de los programas?

- **Tipos fuertes:** Java previene errores en tiempo de compilación
- **Chequeo de límites:** Arrays evitan corrupción de memoria
- **Excepciones:** Errores se detectan y manejan apropiadamente
- **Abstracción:** TADs ocultan detalles de implementación complejos